Cost Trade-offs of Reasoning and Non-Reasoning Large Language Models in Text-to-SQL

Saurabh Deochake, SentinelOne
Dr. Debajyoti Mukhopadhyay, WIDiCoReL Research Lab

Beyond SQL 2026 Workshop @ ICDE 2026

Motivation: The Gap Between Accuracy and Cost

Text-to-SQL benchmarks focus on correctness but production deployments pay per byte

Accuracy milestone: State-of-the-art exceeds 85% on Spider, 75% on BIRD

Cloud pricing reality: BigQuery charges $6.25/TB scanned, regardless of execution speed

BIRD's VES metric: Measures wall-clock time on local SQLite — no cloud cost awareness

The gap: A query scanning an entire table may execute quickly via parallelization but cost 20× more

Enterprise risk: One inefficient query pattern × thousands of users = massive operational cost

Key Insight

Execution time correlates weakly with query cost ($r = 0.16$, $R^2 = 0.026$)

Optimizing for speed ≠ Optimizing for cost

Existing efficiency metrics fail to capture cloud cost dynamics.

"No prior work has systematically measured cloud compute costs for LLM-generated SQL queries."

Methodology

Controlled experiment measuring cloud compute costs of LLM-generated SQL

Platform

- Google BigQuery (US multi-region)
- Caching disabled (useQueryCache=false)
- INFORMATION_SCHEMA.JOBS for metrics
- 300s execution timeout

Workload

- StackOverflow dataset (230 GB)
- 30 NL questions (10 S / 10 M / 10 C)
- 597M rows across 7 tables
- Largest: post_history at 113 GB

Prompt Design

- Zero-shot (no optimization hints)
- Identical prompt across all 6 models
- Full schema with types & FKs
- Default temperature settings

Models Under Evaluation

6 LLMs from 3 major vendors, evenly split between reasoning and standard

Reasoning Models (Extended Thinking)

Opus 4.5 Thinking
Anthropic — Most capable model with explicit reasoning traces

GPT-5.2 High Reasoning
OpenAI — Configured for extended reasoning depth

Gemini 3 Pro Thinking
Google — Enhanced reasoning and thinking capabilities

Standard Models (Speed-Optimized)

Sonnet 4.5
Anthropic — Flagship model balancing capability & efficiency

GPT-5.1
OpenAI — Optimized for throughput and latency

Gemini 3 Flash
Google — Lightweight model for fast inference

StackOverflow Dataset Schema

Table	Rows	Size (GB)
post_history	152M	113
posts_questions	23M	37
posts_answers	34M	29
comments	87M	16
votes	236M	7
users	19M	3
badges	46M	2
Total	597M	230

Cost Metrics Measured

- **Bytes Processed (Bp):** Primary cost driver
- **Bytes Shuffled (Bs):** Data moved between workers
- **Bytes Spilled to Disk (Bd):** Memory pressure indicator
- **Slot Seconds (S):** Compute resources consumed
- **Execution Time (T):** Wall-clock time
- **Estimated Cost:** $C = (Bp / 10^{12}) \times \$6.25$

Results at a Glance

180 query executions across 6 LLMs on 230 GB StackOverflow dataset

44.5%
Fewer bytes processed by reasoning models

3.4×
Cost difference (best vs. worst model)

96.7-100%
Correctness across all models

r = 0.16
Time-cost correlation (weak)

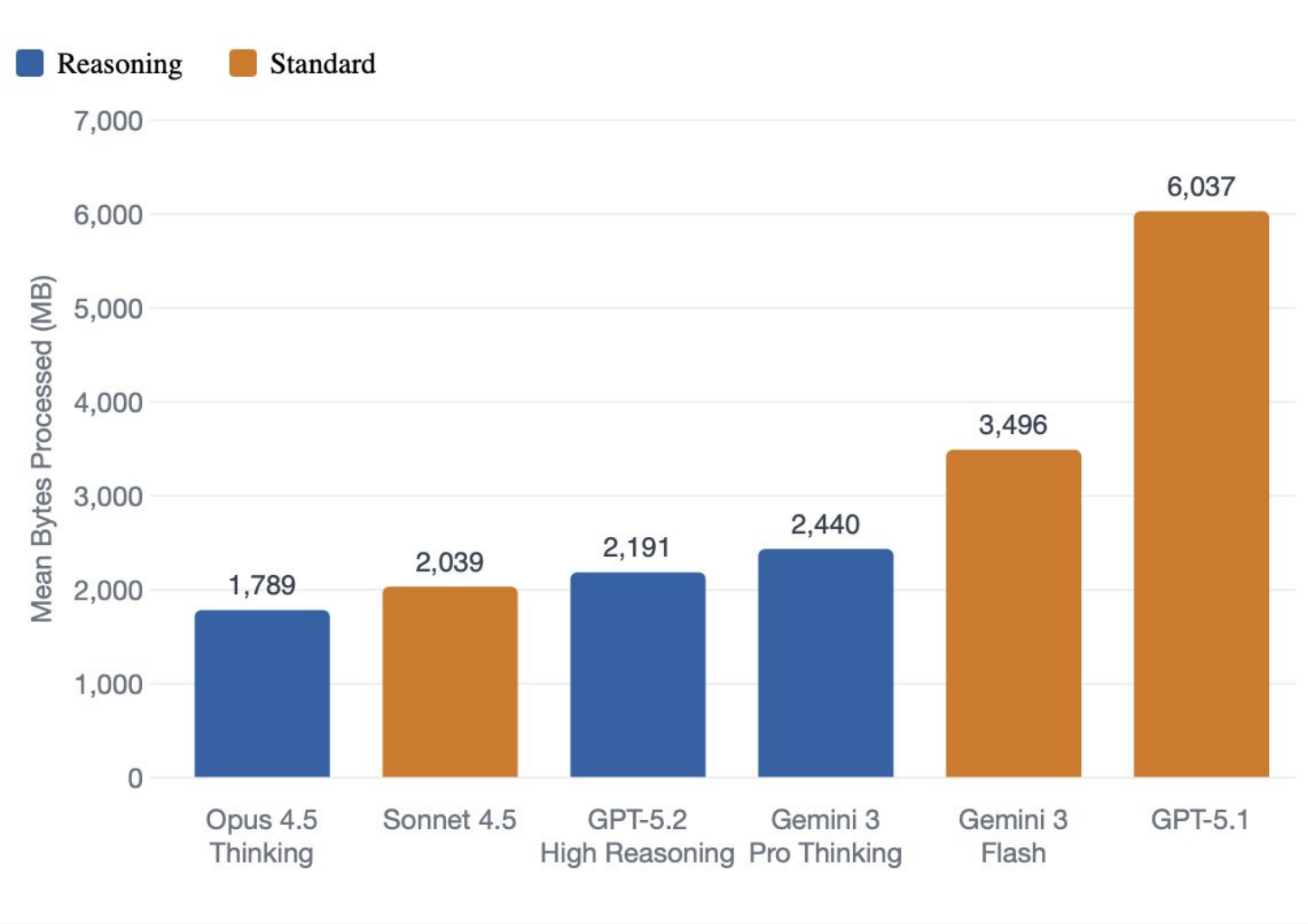
Primary Cost & Performance Metrics (Per Query Average)

Model	Mean Bytes (MB)	Median Bytes (MB)	Mean Time (s)	Est. Cost ($)
Opus 4.5 Thinking	1,789	1,370	3.09	$0.0112
Sonnet 4.5	2,039	1,625	4.36	$0.0127
GPT-5.2 High Reasoning	2,191	1,270	2.65	$0.0137
Gemini 3 Pro Thinking	2,440	1,300	2.82	$0.0153
Gemini 3 Flash	3,496	1,190	3.20	$0.0219
GPT-5.1	6,037	1,365	4.27	$0.0377

Note: Median bytes are consistent (1,190-1,625 MB) — mean differences driven by high-cost outlier queries

Primary Cost Metrics: Bytes Processed by Model

Reasoning models (blue) consistently process fewer bytes than standard models (orange)



Why does Opus 4.5R win?

- Extended thinking lets it plan query structure before generating SQL
- Applied partition filters in 89% of applicable queries
- Used explicit column lists in 97% of queries, avoiding unnecessary column scans

Why is GPT-5.1 so expensive?

- Optimized for low-latency, commits to SQL structure early
- Selected unnecessary columns (e.g., full post body text in JOINs)
- Missed partition filters on the 113 GB post_history table
- 4 queries exceeded 5 GB; worst hit 36.6 GB (20× best model's avg)

Reasoning vs. Standard Models

Statistically significant cost advantage for reasoning models (p = 0.003, Cohen's d = 0.52)

Why does reasoning help?

Standard models generate SQL token-by-token, committing early to suboptimal structures. Reasoning models "think first" — considering predicate pushdown, column pruning, and join reordering before writing SQL.

Evidence from generated SQL:

Partition filters: 89% vs. 67%

Explicit column lists: 97% vs. 93%

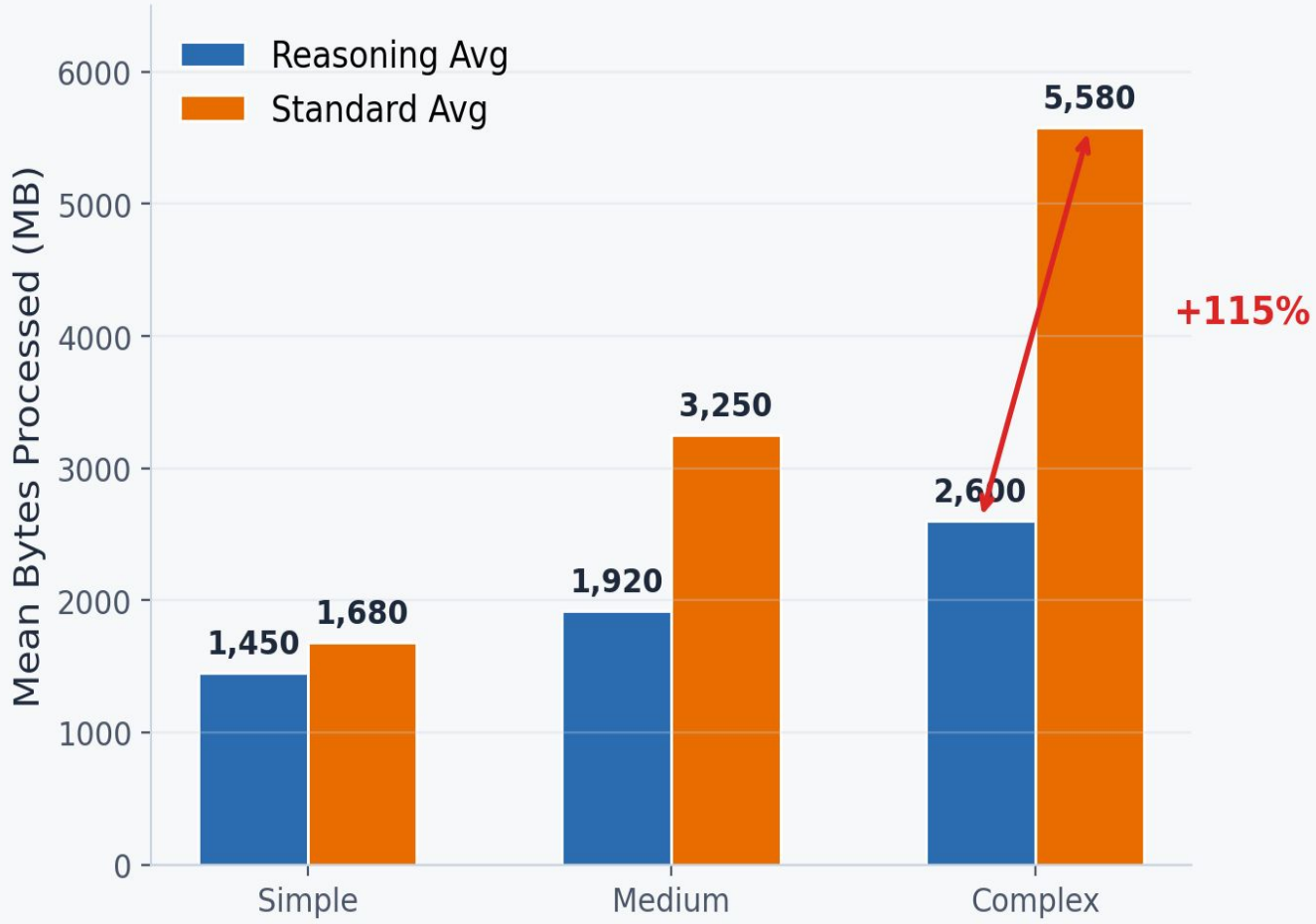
Fewer full-table scans on 113 GB tables

Statistical Validation

Mann-Whitney U:	U = 2,847, p = 0.003	**Effect Size:**	Cohen's d = 0.52 (medium)
Median Comparison:	1,313 MB (reasoning) vs. 1,393 MB (standard)	**Cost Savings:**	44.5% fewer bytes and 44.4% lower cost ($0.0134 vs. $0.0241/query)

Cost Disparities by Query Complexity

The advantage of reasoning models grows dramatically with query complexity



Why does the gap widen with complexity?

Simple queries (single-table) have limited optimization space — both model types pick similar plans.

Complex queries have many decision points: join order, column selection, filter placement. Each suboptimal choice compounds. Reasoning models plan the full structure first; standard models commit token-by-token and accumulate errors.

Simple: +16%

Limited optimization space for single-table queries

Medium: +70%

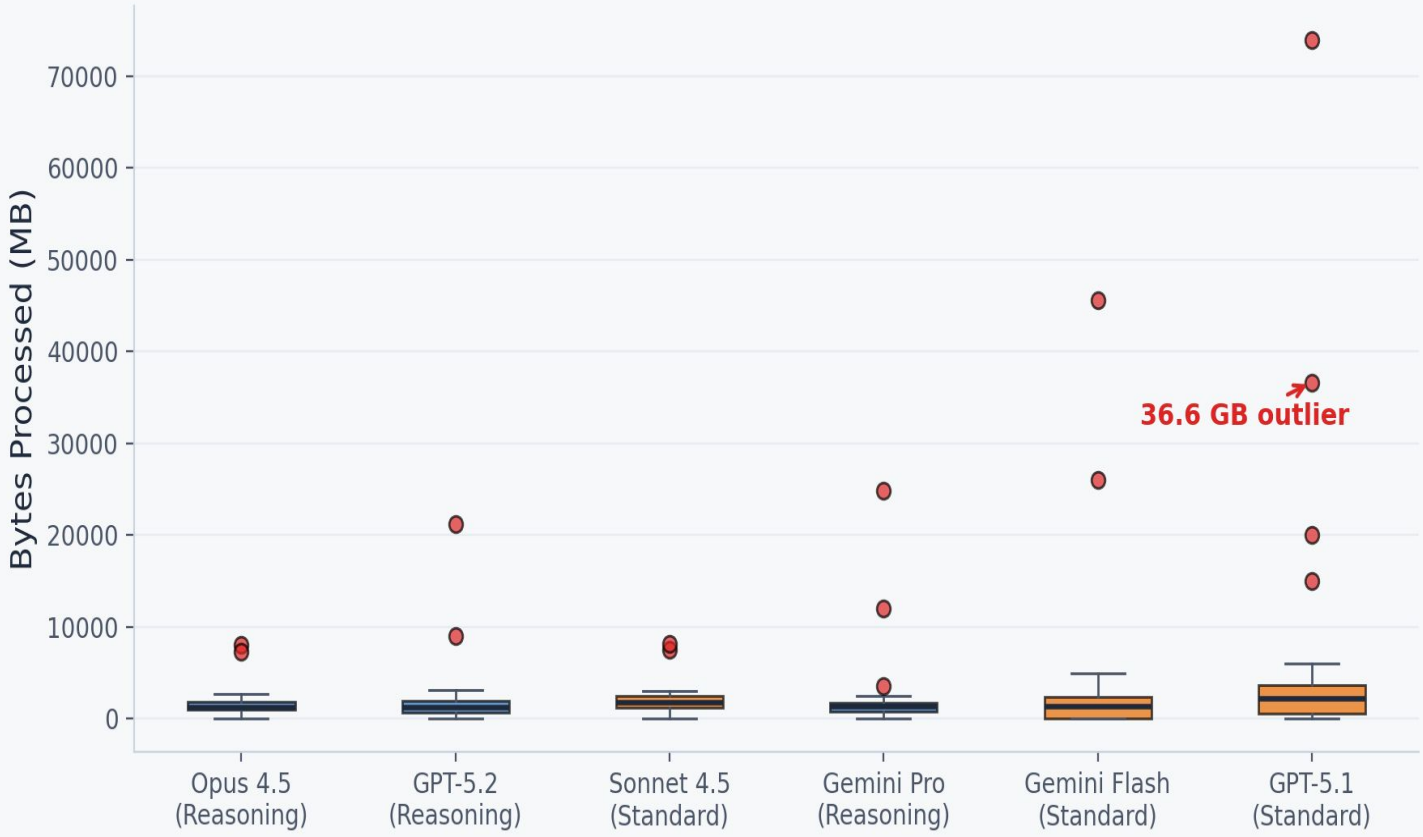
Multi-table joins amplify suboptimal column and filter choices

Complex: +115%

Subqueries + window functions + 4+ joins compound errors

Cost Variance and Outliers

Standard models exhibit extreme cost unpredictability



Cost Consistency Metrics

Model	Std Dev (MB)	CV	> 5 GB
GPT-5.1	11,659	1.93	4
Gemini Flash	6,462	1.85	2
Gemini Pro Thinking	5,467	2.24	1
GPT-5.2 HR	3,032	1.38	1
Opus 4.5 Thinking	2,823	1.58	1
Sonnet 4.5	2,864	1.40	1

Why are standard models less predictable?

Without a planning phase, standard models occasionally "miss" critical optimizations entirely, forgetting partition filters or selecting all columns. These misses create catastrophic outliers that skew averages while medians stay similar.

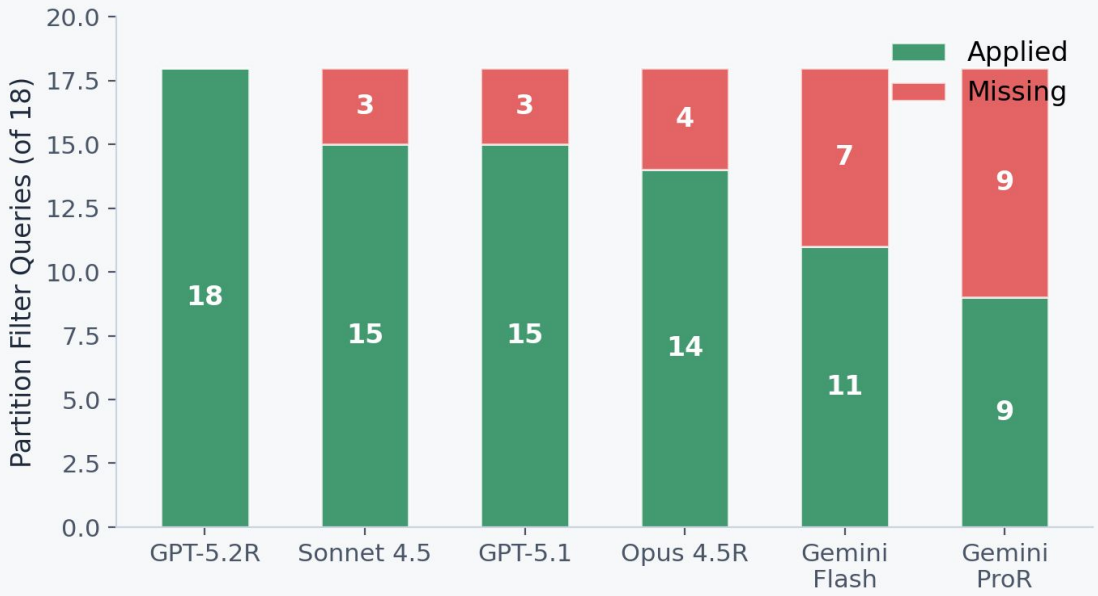
GPT-5.1: std dev 11,659 MB ≈ 2× its mean

Worst query: 36.6 GB (selected full post body in JOIN)

Reasoning CV: 1.38-1.58 vs. Standard CV: 1.85-1.93

SQL Inefficiency Patterns in LLM-Generated Queries

Query formulation problems that the database optimizer cannot fix



SELECT *

Scans all columns in columnar storage. Only OpenAI models (1 each).

Cross Join

Cartesian products from missing join conditions. GPT-5.2R & Gemini Flash.

Missing Partition Filters

Most common: up to 50% miss rate (Gemini Pro: 9/18).

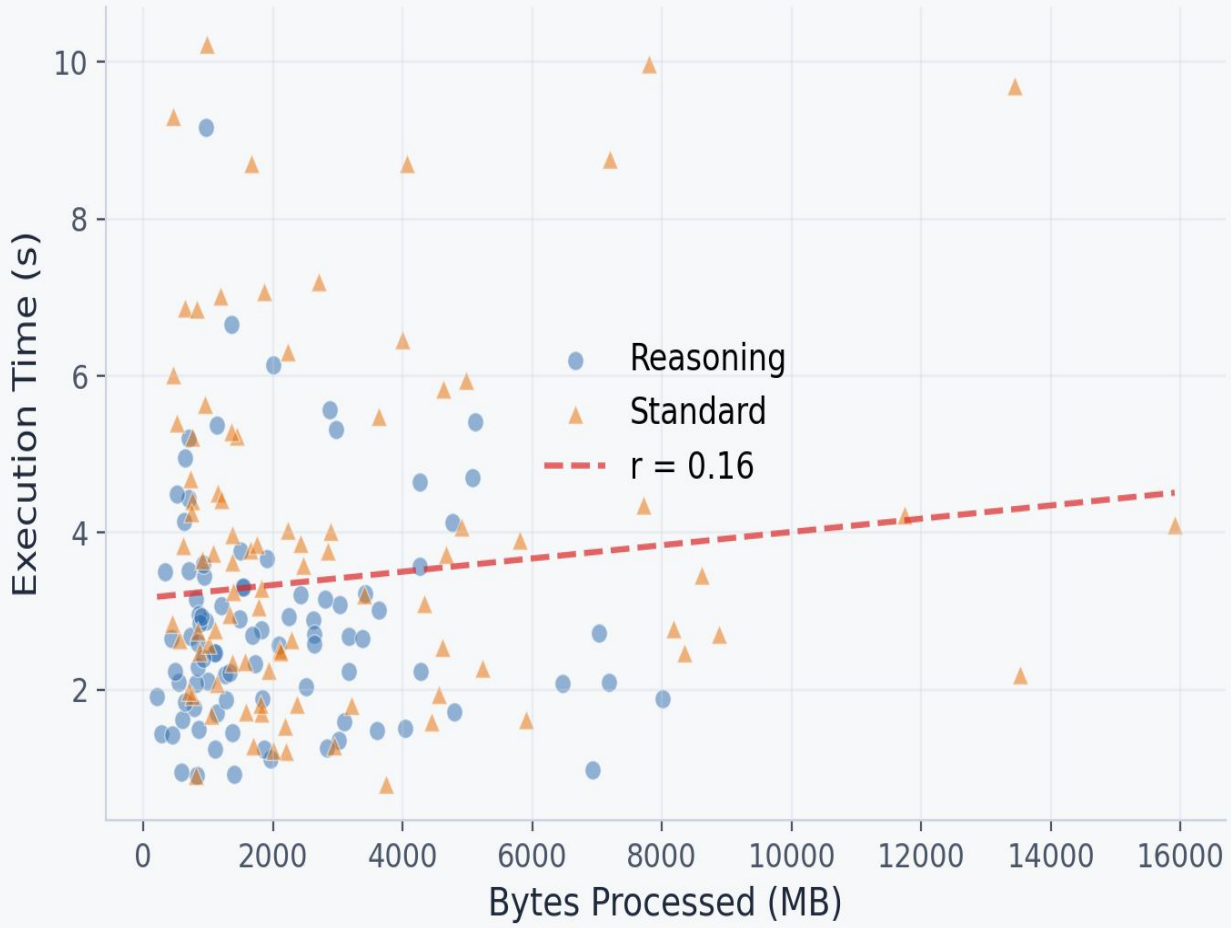
Excessive CTEs

Can inhibit predicate pushdown. OpenAI avg 0.60-0.63/query.

Why Can't BigQuery's Optimizer Fix This?

The optimizer operates on the physical query plan after SQL is parsed. It can reorder joins and push predicates within the plan, but it cannot add semantically meaningful filters absent from the original SQL. If a query omits a date predicate on a partitioned table, the optimizer must scan all partitions. These are query formulation issues — the choice of LLM directly impacts cost.

Speed ≠ Cost: Correlation Analysis



Correlation Matrix

Metric Pair	Pearson r	Strength
Bytes vs. Exec Time	0.16	Weak
Bytes vs. Shuffled	0.34	Moderate
Bytes vs. SQL Length	0.05	None
Time vs. Slot Seconds	0.64	Strong

Why doesn't speed predict cost?

- BigQuery massively parallelizes: a full scan of a 113 GB table uses thousands of slots and finishes in 3 seconds - but you pay for 113 GB. A complex computation on 500 MB takes longer but costs 200× less.
- BIRD's VES metric measures wall-clock time on local SQLite, which lacks this parallelization. VES rewards speed, not cost efficiency.

Surprising finding:

Bytes vs. SQL length r = 0.05 - verbose SQL with explicit columns and proper filters scans far less than a concise SELECT*

Deployment Guidelines for Cost-Sensitive Environments

1

Prefer reasoning models for analytical workloads

Despite higher inference costs, reasoning models generate queries that are 44.5% cheaper to execute. For data-intensive applications, execution cost savings likely exceed the additional inference cost.

2

Implement cost guardrails

Deploy query cost estimation and rejection thresholds before execution. Integrate FinOps practices such as automated budget enforcement and pre-deployment cost estimation to prevent runaway queries.

3

Monitor for SQL anti-patterns

Build automated detection for SELECT *, missing partition filters, and unintended CROSS JOINs. Catch costly queries before they reach the data warehouse.

4

Do not use execution time as a cost proxy

The weak correlation ($r = 0.16$) between time and bytes processed means fast queries can still be expensive. Measure and optimize bytes processed directly.

Limitations & Future Work

Limitations

Single platform (BigQuery) with one dataset (StackOverflow, normalized schema)

30-query benchmark is smaller than Spider (1,034) or BIRD (12,751)

Single execution per model-query pair (BigQuery cost is deterministic, but LLM generation is stochastic)

Reasoning models are also flagship models — observed differences may partially reflect overall capability

Zero-shot prompting only; prompt engineering may shift results

Future Work

Multi-platform evaluation: Snowflake, Redshift, Databricks to test generalizability

Cost-aware prompting: "Minimize bytes scanned" — can prompts improve cost efficiency?

RL for cost optimization: Incorporate cloud cost as a reward signal in RL-based Text-to-SQL

Multi-run generation analysis: Quantify generation-side variance with non-zero temperature

Real-time cost estimation: Integrate SQL generation with query cost predictors for pre-execution guardrails

Thank You!

Questions & Discussion

Saurabh Deochake | SentinelOne | saurabh.deochake@sentinelone.com

Debajyoti Mukhopadhyay | WIDiCoReL Research Lab | debajyoti.mukhopadhyay@gmail.com

Data & Artifacts: https://doi.org/10.5281/zenodo.18070764